

AMRITA VISHWA VIDYAPEETHAM – CHENNAI CAMPUS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
23CSE302 – COMPUTER NETWORKS

EXPERIMENT 3

**IMPLEMENTATION OF SIMPLE CLIENT- SERVER FILE
TRANSFER PROTOCOL USING JAVA AND CISCO PACKET
TRACER**

Name: Chetan Kumar G		Max Marks (10)
Roll No.: CH.SC.U4CSE24109	Section: CSE-B	
Date of Exp: 05/07/2026	Date of Submission: 05/07/2026	

OBJECTIVES:

- 1. Students will have exposure to FTP using Java**
- 2. Students will have an understanding how file transfer can be simulated between client and FTP server in Cisco Packet Tracer using FTP protocol**

A. Implementation of Simple Client-Server File Transfer Protocol in Java

AIM

To implement a basic File Transfer Protocol (FTP) using socket programming in Java, where a server sends a file to a client upon connection.

SYSTEM AND SOFTWARE REQUIREMENTS

Software – VS Code

Libraries – java.io.* and java.net.*

PROCEDURE

A. Server

- 1) Import Required Libraries:
 - Import the java.io.* and java.net.* libraries.
- 2) Create the Server Socket:
 - Use the ServerSocket class to create a server socket on port 8000.
- 3) Accept Client Connections:
 - Use the accept() method to listen for and accept client connection requests.
- 4) Read and Send File:
 - Use a FileInputStream to read the file specified in the command line arguments.
 - Use an OutputStreamWriter to send the file data to the client.
- 5) Close Connections:
 - Close the socket and file input stream after the file transfer is complete.

B. Client

- 1) Import Required Libraries:
 - Import the java.io.* and java.net.* libraries.

2) Create the Client Socket:

- Use the Socket class to create a client socket and connect to the server at the local host on port 8000.

3) Receive and Write File:

- Use a FileOutputStream to write the received file data to a file specified in the command line arguments.
- Use an InputStreamReader to read the file data sent by the server.

4) Close Connections:

- Close the socket and file output stream after the file transfer is complete.

JAVA CODE

A. Server

```
import java.io.*;
import java.net.*;

public class Server {

    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(port: 8000);
            System.out.println("Server is waiting for client...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");
            FileInputStream fis = new FileInputStream(args[0]);
            OutputStream os = socket.getOutputStream();
            int ch;
            while ((ch = fis.read()) != -1) {
                os.write(ch);
            }
            System.out.println("File sent successfully.");
            fis.close();
            os.close();
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

B. Client

```
Run | Debug
import java.io.*;
import java.net.*;
public class Client {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket(host: "localhost", port: 8000);
            System.out.println(x: "Connected to server.");
            InputStream is = socket.getInputStream();
            FileOutputStream fos = new FileOutputStream(args[0]);
            int ch;
            while ((ch = is.read()) != -1) {
                fos.write(ch);
            }
            System.out.println(x: "File received successfully.");
            fos.close();
            is.close();
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT SNAPSHOT

```
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet-3> javac Server.java
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet-3> java Server sample.txt
Server is waiting for client...
Client connected.
File sent successfully.
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet-3> █
```

```
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet-3> javac Client.java
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet-3> java Client received.txt
Connected to server.
File received successfully.
PS C:\Users\chetan kumar g\OneDrive - Amrita Vishwa Vidyapeetham- Chennai Campus\V
Sem\Computer Networks\Lab Activity\Worksheet-3> █
```

RESULT

The File Transfer Protocol (FTP) was successfully implemented using Java socket programming. The server established a connection with the client on port 8000 and transmitted the specified file. The client successfully received the file and stored it in the destination location without any data loss.

INFERENCE

- Socket programming enables reliable communication between a server and a client over a network.
- A ServerSocket listens for incoming client requests, while a Socket establishes the connection.
- File data can be transferred efficiently using Java input and output streams (FileInputStream and FileOutputStream).
- This experiment demonstrates the basic working principle of the File Transfer Protocol (FTP) and client-server communication in Java.

B. IMPLEMENTATION OF FTP USING CISCO PACKET TRACER

AIM

To implement a File Transfer Protocol (FTP) using Cisco Packet Tracer to transfer files between a client and a server within a simulated network environment.

SYSTEM AND SOFTWARE REQUIREMENTS

Software – Cisco Packet Tracer

Components used:

- 1) End Devices
 - PC-PT
 - Server -PT
- 2) Network Devices
 - Router 2911
 - Switch 2950-24
- 3) Connections
 - Copper Straight-Through
 - Console Cable

PROCEDURE

To implement a File Transfer Protocol (FTP) using Cisco Packet Tracer to transfer files between a client and a server, follow these steps:

Step 1: Open Cisco Packet Tracer

- 1) Launch Cisco Packet Tracer on your computer.

Step 2: Add Devices to the Workspace

- 1) Add two PCs:
 - Click on the "End Devices" icon in the bottom left corner.
 - Select "PC" and drag two PCs into the workspace.

- 2) Add a server:
 - Click on the "End Devices" icon in the bottom left corner.
 - Select "Server" and drag a server into the workspace.
- 3) Add a Switch:
 - Click on the "Switches" icon in the bottom left corner.
 - Select a switch (2950-24) and drag it into the workspace.
- 4) Add a Router:
 - Click on the "Routers" icon in the bottom left corner.
 - Select a router (2911) and drag it into the workspace.

Step 3: Connect the Devices

- 1) Select the Connection Tool:
 - Click on the "Connections" icon (lightning bolt) in bottom left corner.
- 2) Connect the PCs to the Switch:
 - Click on a PC and then click on the switch to connect them with a copper straight-through cable.
 - Repeat the process for the second PC.
- 3) Connect the Switch to the Router:
 - Click on the switch and then click on the router to connect them with a copper straight-through cable.
- 4) Connect the Router to the Server:
 - Click on the router and then click on the Server to connect them with a copper straight-through cable.
- 5) Connect a PC to the Router:
 - Click on the PC and then click on the router to connect them with a console cable.

Step 4: Configure IP Addresses:

- 1) Configure the first PC (Faculty PC):
 - Open the Faculty PC, click on Desktop, select IP Configuration
 - Choose Static to manually enter the following,

IPv4 Address: 192.168.1.2

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

2) Configure the second PC (Student PC):

- Open the Student PC, click on Desktop, select IP Configuration
- Choose Static to manually enter the following,

IPv4 Address: 192.168.1.3

Subnet Mask: 255.255.255.0

Default Gateway: 192.168.1.1

3) Configure the router (AVV Chennai Router):

- On the router, go to the Config tab and select the GigabitEthernet0/0 interface for LAN interface. Under IP Configuration, manually enter the following,

IPv4 Address: 192.168.1.1

Subnet Mask: 255.255.255.0

- On the router, go to the Config tab and select the GigabitEthernet0/1 interface for WAN interface. Under IP Configuration, manually enter the following,

IPv4 Address: 10.0.0.1

Subnet Mask: 255.0.0.0

4) Configure the server (Amritapuri File Server):

- Open the Amritapuri File Server, click on Desktop, select IP Configuration
- Choose Static to manually enter the following,

IPv4 Address: 10.0.0.2

Subnet Mask: 255.0.0.0

Default Gateway: 10.0.0.1

Step 5: FTP Server Configuration

- 1) On the Amritapuri File Server, go to the Services tab and select FTP.
- 2) Ensure the FTP service is turned ON.
- 3) Under User Setup, add a new user with the following details:

Username: Amrita

Password: amma@123

4) Enable the following permissions, Read write and Load (RWL)

Step 6: Testing Connectivity

- 1) Ping the Amritapuri File Server from the Faculty PC and Student PC to ensure network connectivity.
- 2) Example: To check the network connectivity of Faculty PC,
 - Open the Faculty PC, click on Desktop, select command prompt and enter the following
 - To test the network connectivity between the Faculty PC and the Student PC: **Ping 192.168.1.3**
 - To test the network connectivity between the Faculty PC and the LAN interface of the 2911 Chennai Router: **Ping 192.168.1.3**
 - To test the network connectivity between the Faculty PC and the WAN interface of the 2911 Chennai Router: **Ping 10.0.0.1**
 - To test the network connectivity between the Faculty PC and the Amritapuri File Server on the WAN network: **Ping 10.0.0.2**

Step 7: FTP Client Configuration- Upload a file from Faculty PC

- 1) On the Faculty PC, navigate to the Desktop tab and open the Text Editor.
- 2) Create a new text file, add the desired content, and save the file with the name `msg.txt`.
- 3) Navigate to the Desktop tab and open the Command Prompt:
 - To initiate an FTP connection from the Faculty PC to the Amritapuri File Server enter: **ftp 10.0.0.2**
 - Enter the valid FTP credentials to log in to the FTP server.

Username: Amrita Password:

amma@123

- To list the contents of the current directory on the FTP server enter:
ftp>dir
- To upload a file named msg.txt from the Faculty PC to the current directory on the FTP server enter: **ftp>put msg.txt**

Step 8: FTP Client Configuration- Download the file in Student PC

1) On the Student PC, navigate to the Desktop tab and open the Command Prompt:

- To initiate an FTP connection from the Faculty PC to the Amritapuri File Server enter: **ftp 10.0.0.2**
- Enter the valid FTP credentials to log in to the FTP server.

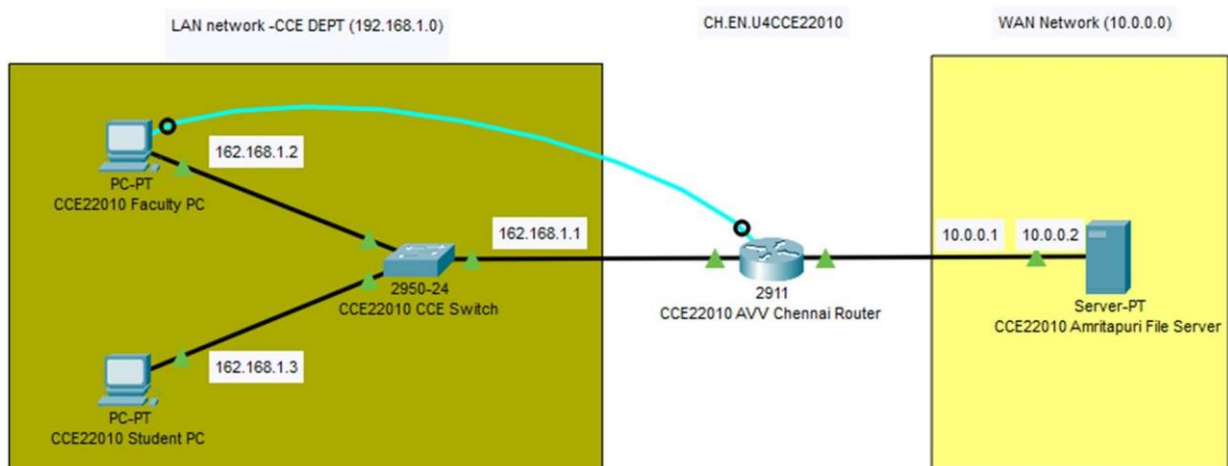
Username: Amrita Password:

amma@123

- To download the file named msg.txt from the FTP server to the Student PC's current directory enter: **ftp>get msg.txt**
- To terminate the FTP session and returns the user to the command prompt enter: **ftp>quit**
- To list the contents of the current directory on the Student PC again to verify that msg.txt has been successfully downloaded from the FTP server enter: **dir**

By following these steps, we can successfully implement a File Transfer Protocol (FTP) using Cisco Packet Tracer to transfer files between a client and a server.

NETWORK TOPOLOGY DIAGRAM



OUTPUT

A. Network Connectivity test of Faculty PC with other devices:

1) Faculty PC and Student PC:

```
C:\>ping 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data:

Reply from 192.168.1.2: bytes=32 time<1ms TTL=128
Reply from 192.168.1.2: bytes=32 time<1ms TTL=128
Reply from 192.168.1.2: bytes=32 time=8ms TTL=128
Reply from 192.168.1.2: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 8ms, Average = 2ms

C:\>
```

2) Faculty PC and LAN interface of the 2911 Chennai Router:

```
C:\>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:

Reply from 192.168.1.1: bytes=32 time=16ms TTL=255
Reply from 192.168.1.1: bytes=32 time<1ms TTL=255
Reply from 192.168.1.1: bytes=32 time=1ms TTL=255
Reply from 192.168.1.1: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 16ms, Average = 4ms
```

3) Faculty PC and the WAN interface of the 2911 Chennai Router:

```
C:\>ping 10.0.0.1

Pinging 10.0.0.1 with 32 bytes of data:

Reply from 10.0.0.1: bytes=32 time<1ms TTL=255
Reply from 10.0.0.1: bytes=32 time<1ms TTL=255
Reply from 10.0.0.1: bytes=32 time<1ms TTL=255
Reply from 10.0.0.1: bytes=32 time<1ms TTL=255

Ping statistics for 10.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

4) Faculty PC and the Amritapuri File Server on the WAN network:

```
C:\>ping 10.0.0.2

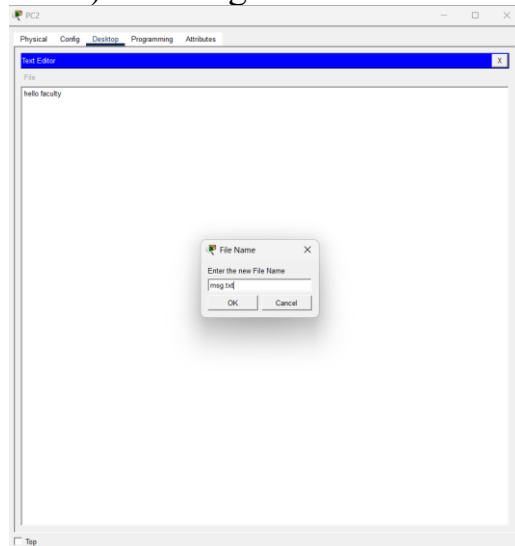
Pinging 10.0.0.2 with 32 bytes of data:

Request timed out.
Reply from 10.0.0.2: bytes=32 time=1ms TTL=127
Reply from 10.0.0.2: bytes=32 time=1ms TTL=127
Reply from 10.0.0.2: bytes=32 time<1ms TTL=127

Ping statistics for 10.0.0.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

B. FTP Client Configuration: Uploading a file from Faculty PC

1) Creating a file name named “msg.txt:



```
C:\>dir

Volume in drive C has no label.
Volume Serial Number is 5E12-4AF3
Directory of C:\

1/1/1970    5:30 PM           13      msg.txt
1/1/1970    5:30 PM           26      sampleFile.txt
               39 bytes           2 File(s)
```

2) Initiate connection to FTP and login:

```
ftp 10.0.0.2
Trying to connect...10.0.0.2
Connected to 10.0.0.2
220- Welcome to PT Ftp server
Username:Amrita
331- Username ok, need password
Password:*****
230- Logged in
(passive mode On)
ftp>
```

3) List the contents of the current directory on the FTP server:

```
ftp 10.0.0.2
Trying to connect...10.0.0.2
Connected to 10.0.0.2
220- Welcome to PT Ftp server
Username:Amrita
331- Username ok, need password
Password:*****
230- Logged in
(passive mode On)
ftp>dir

Listing /ftp directory from 10.0.0.2:
0   : asa842-k8.bin                5571584
1   : asa923-k8.bin                30468096
2   : c1841-advipservicesk9-mz.124-15.T1.bin  33591768
3   : c1841-ipbase-mz.123-14.T7.bin  13832032
4   : c1841-ipbasek9-mz.124-12.bin  16599160
5   : c1900-universalk9-mz.SPA.155-3.M4a.bin  33591768
6   : c2600-advipservicesk9-mz.124-15.T1.bin  33591768
7   : c2600-i-mz.122-28.bin        5571584
8   : c2600-ipbasek9-mz.124-8.bin  13169700
9   : c2800nm-advipservicesk9-mz.124-15.T1.bin  50938004
10  : c2800nm-advipservicesk9-mz.151-4.M4.bin  33591768
11  : c2800nm-ipbase-mz.123-14.T7.bin  5571584
12  : c2800nm-ipbasek9-mz.122-8.bin  15522644
13  : c2900-universalk9-mz.SPA.155-3.M4a.bin  33591768
14  : c2950-i6q412-mz.121-22.EA4.bin  3058048
15  : c2950-i6q412-mz.121-22.EA8.bin  3117390
16  : c2960-lanbase-mz.122-25.FX.bin  4414921
17  : c2960-lanbase-mz.122-25.SEE1.bin  4670455
18  : c2960-lanbasek9-mz.150-2.SE4.bin  4670455
19  : c3560-advipservicesk9-mz.122-37.SE1.bin  8662192
20  : c3560-advipservicesk9-mz.122-46.SE.bin  10713279
21  : c800-universalk9-mz.SPA.152-4.M4.bin  33591768
22  : c800-universalk9-mz.SPA.154-3.M6a.bin  83029236
23  : cat3k_caa-universalk9.16.03.02.SPA.bin  505532849
24  : cgr1000-universalk9-mz.SPA.154-2.CG  159487552
25  : cgr1000-universalk9-mz.SPA.156-3.CG  184530138
26  : ie9k_iosxe.17.09.04.SPA.bin  596133776
27  : ir800-universalk9-bundle.SPA.156-3.M.bin  160968869
28  : ir800-universalk9-mz.SPA.155-3.M  61750062
29  : ir800-universalk9-mz.SPA.156-3.M  63753767
30  : ir800_yocto-1.7.2.tar        2877440
31  : ir800_yocto-1.7.2_python-2.7.3.tar  6912000
32  : ir8340-mono-universalk9_iot.17.08.01a.SPA.pkg  685645824
33  : msg.txt                      53
34  : pt1000-i-mz.122-28.bin        5571584
35  : pt3000-i6q412-mz.121-22.EA4.bin  3117390
```

4) Upload the msg.txt file in FTP directory and exit:

```
ftp>put msg.txt

Writing file msg.txt to 10.0.0.2:
File transfer in progress...

[Transfer complete - 13 bytes]

13 bytes copied in 0.06 secs (216 bytes/sec)
ftp>
```

C. FTP Client Configuration: Downloading a file from Student PC

1) Initiate connection to FTP and login:

```
C:\>ftp 10.0.0.2
Trying to connect...10.0.0.2
Connected to 10.0.0.2
220- Welcome to PT Ftp server
Username:Amrita
331- Username ok, need password
Password:*****
230- Logged in
(passive mode On)
ftp>
```

2) Download the file named msg.txt from the FTP server to the Student PC:

```
C:\>ftp 10.0.0.2
Trying to connect...10.0.0.2
Connected to 10.0.0.2
220- Welcome to PT Ftp server
Username:Amrita
331- Username ok, need password
Password:*****
230- Logged in
(passive mode On)
ftp>get msg.txt

Reading file msg.txt from 10.0.0.2:
File transfer in progress...

[Transfer complete - 13 bytes]

13 bytes copied in 0.001 secs (13000 bytes/sec)
ftp>
```

3) List the contents of the current directory on the Student PC:

3 bytes copied in 0.001 secs (13000 bytes/sec)

tp>dir

isting /ftp directory from 10.0.0.2:

:	asa842-k8.bin	5571584
:	asa923-k8.bin	30468096
:	c1841-advipservicesk9-mz.124-15.T1.bin	33591768
:	c1841-ipbase-mz.123-14.T7.bin	13832032
:	c1841-ipbasek9-mz.124-12.bin	16599160
:	c1900-universalk9-mz.SPA.155-3.M4a.bin	33591768
:	c2600-advipservicesk9-mz.124-15.T1.bin	33591768
:	c2600-i-mz.122-28.bin	5571584
:	c2600-ipbasek9-mz.124-8.bin	13169700
:	c2800nm-advipservicesk9-mz.124-15.T1.bin	50938004
0 :	c2800nm-advipservicesk9-mz.151-4.M4.bin	33591768
1 :	c2800nm-ipbase-mz.123-14.T7.bin	5571584
2 :	c2800nm-ipbasek9-mz.124-8.bin	15522644
3 :	c2900-universalk9-mz.SPA.155-3.M4a.bin	33591768
4 :	c2950-i6q412-mz.121-22.EA4.bin	3058048
5 :	c2950-i6q412-mz.121-22.EA8.bin	3117390
6 :	c2960-lanbase-mz.122-25.FX.bin	4414921
7 :	c2960-lanbase-mz.122-25.SEE1.bin	4670455
8 :	c2960-lanbasek9-mz.150-2.SE4.bin	4670455
9 :	c3560-advipservicesk9-mz.122-37.SE1.bin	8662192
0 :	c3560-advipservicesk9-mz.122-46.SE.bin	10713279
1 :	c800-universalk9-mz.SPA.152-4.M4.bin	33591768
2 :	c800-universalk9-mz.SPA.154-3.M6a.bin	83029236
3 :	cat3k_caa-universalk9.16.03.02.SPA.bin	505532849
4 :	cgr1000-universalk9-mz.SPA.154-2.CG	159487552
5 :	cgr1000-universalk9-mz.SPA.156-3.CG	184530138
6 :	ie9k_iosxe.17.09.04.SPA.bin	596133776
7 :	ir800-universalk9-bundle.SPA.156-3.M.bin	160968869
8 :	ir800-universalk9-mz.SPA.155-3.M	61750062
9 :	ir800-universalk9-mz.SPA.156-3.M	63753767
0 :	ir800_yocto-1.7.2.tar	2877440
1 :	ir800_yocto-1.7.2_python-2.7.3.tar	6912000
2 :	ir8340-mono-universalk9_iot.17.08.01a.SPA.pkg	685645824
3 :	msg.txt	13
4 :	pt1000-i-mz.122-28.bin	5571584
5 :	pt3000-i6q412-mz.121-22.EA4.bin	3117390

trn\

RESULT

- 1) Successful FTP connection from client PC's
- 2) Successful File transfer between client PC's through FTP protocol

INFERENCE

The experiment demonstrated the working of the File Transfer Protocol (FTP) in a simulated network environment. Proper IP addressing, routing, and FTP server configuration enabled reliable file sharing between different hosts. The activity provided practical knowledge of FTP authentication, file upload/download operations, and network connectivity testing using Cisco Packet Tracer.

Lab Scenario: Secure Document Sharing in a Corporate Environment

Scenario Title: *“Designing a File Sharing System for Secure Inter-Department Communication”*

Background:

Your company, **CyberWave Pvt. Ltd.**, has multiple departments working in different parts of the campus. The management has instructed the IT team to enable **secure and efficient file sharing** between client systems and a centralized document server. This must be done in two phases:

Part A: Java-Based File Transfer System

Task:

You are tasked with developing a **basic file transfer protocol using Java socket programming**. The goal is to:

- Allow a **client** to connect to the **file server**.
- Request a file by name.
- Download the file if it exists on the server.
- Handle the case where the requested file is not found.

Requirements:

1. Create a **Java server application** that:
 - Listens on a port.
 - Waits for a client to request a file.
 - Sends the file if available, or returns an error message.
2. Create a **Java client application** that:
 - Connects to the server.
 - Requests a specific file.
 - Downloads and saves the file locally.

Q1: What file did you request from the server, and what was the size of the transferred file?

Q2: Show and explain how your client handles a "file not found" situation.

Part B: FTP Network Simulation in Cisco Packet Tracer

Task:

Now simulate the same environment using **Cisco Packet Tracer** by setting up:

- An **FTP server** (using the server module in Packet Tracer).
- At least **two client PCs** from different network segments.
- Proper **IP addressing, routing (if required), and FTP service configuration**.

Each client PC should be able to:

- Connect to the FTP server.
- Upload and download files.

Requirements:

1. Configure the FTP server with login credentials and file permissions.
2. Test the **upload** of a file from PC1 and **download** the same file from PC2.

Q3: What was the IP configuration of the FTP server and clients?

Q4: Demonstrate successful upload and download by showing relevant screenshots and logs.

Q5 (Challenge): What network issues (e.g., incorrect routing, firewall settings) might prevent FTP communication, and how did you resolve them?

Viva Questions:

Question	Company
How does file transfer happen using sockets?	Infosys, Wipro
What happens if the file doesn't exist on the server?	TCS, Dell
How do you simulate this system in Packet Tracer?	Cisco
What are some limitations of FTP?	Palo Alto, IBM